

ARXIV:2604.25917V1 · UIUC / STANFORD / NVIDIA / MIT

RecursiveMAS

Recursive Multi-Agent Systems

Scaling agent collaboration through recursive latent refinement — agents form a loop, passing continuous latent “thoughts” across rounds to progressively deepen solutions.

+8.3%

Avg Accuracy

2.4×

Inference Speedup

75.6%

Fewer Tokens

9

Benchmarks

4

Collab Patterns

LangGraph

create_react_agent

StateGraph

Claude / Qwen / Llama

THE CHALLENGE

Why Recursive Multi-Agent Systems?

"Can agent collaboration itself be scaled through *recursion*?"

LIMITATIONS

- ▶ Single models hit capacity ceilings on complex tasks
- ▶ Text-based MAS: slow, expensive — agents wait for full text generation
- ▶ Each round must re-encode all previous text outputs
- ▶ No shared gradient — agents can't co-evolve as a system

RECURSIVEMAS SOLUTION

- ▶ Agents share latent states — no re-encoding overhead
- ▶ RecursiveLink bridges agents in continuous embedding space
- ▶ Inner-outer loop: gradient flows across all agents jointly
- ▶ Structure-agnostic — works with any collaboration topology

Key insight

Replace vocabulary-space decoding cost $m|V|d_h$ with efficient latent transform md_h^2

ARCHITECTURE

RecursiveLink: Inner & Outer Modules

A lightweight 2-layer residual projection that bridges agents without touching model weights.

Inner Link R_{in} within each agent

Last-layer h → Linear W_1 → GELU →

Linear W_2

+ residual h → Input e_{t+1}

$$R_{in}(h) = h + W_2 \sigma(W_1 h)$$

Maps latent thought back into input embedding space for next forward pass. No vocab projection needed.

Outer Link R_{out} between agents

Agent A_i emb → Linear W_1 → GELU → W_2

+ $W_3 h$ (cross-dim) → Agent A_j emb

$$R_{out}(h) = W_3 h + W_2 \sigma(W_1 h)$$

Extra W_3 handles heterogeneous hidden dimensions across different LLM families (e.g. Qwen → Llama).

RECURSIVE MECHANICS

How the Recursive Loop Works

Each recursion round deepens the collective reasoning. Agents refine outputs using accumulated latent context from previous rounds.

**Inner Loop (per agent)**

Each agent generates m latent thoughts via auto-regressive steps using R_{in} . No text generated until the final round.

Outer Loop (across agents)

After each agent finishes, R_{out} projects its latent thoughts into the next agent's embedding space — direct latent handoff.

Shared Credit Assignment

A single CE loss on final output backpropagates through *all* recursion rounds and all outer links simultaneously.

Stable Gradients

Residual connections in RecursiveLink preserve gradient norms — avoids vanishing gradients across deep recursion.

Sequential Style chain-of-agents

Planner → Critic → Solver

Three agents with complementary roles progressively decompose, judge, and solve.
Each round's Solver output feeds back as context to the Planner.



Planner

Decomposes problem into structured action plan. Refines based on prior round's feedback.

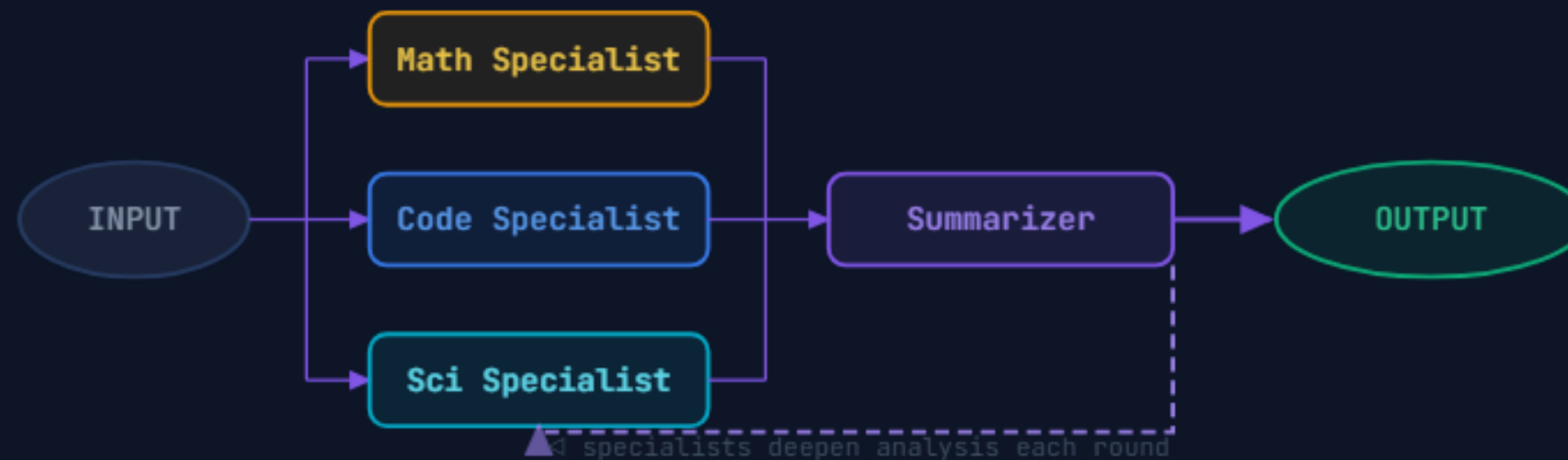
Critic

Identifies logical gaps, missing steps, faulty assumptions. Ensures plan quality before solving.

Mixture Style mixture-of-experts

Domain Specialists → Summarizer

Math, Code, and Science specialists reason over the input in parallel. A Summarizer synthesizes their insights into one unified answer.



Domain Specialists

Math, Code, and Science agents each contribute domain expertise. Run sequentially (parallel in theory).

Summarizer

Synthesizes all specialist views into one coherent answer, resolving contradictions and prioritizing strongest contributions.

Distillation Style expert-to-learner

Expert → Learner

A capable Expert agent guides a smaller, faster Learner. Each round the Learner internalises corrections and becomes more autonomous — efficient knowledge distillation.



Expert Role

Produce comprehensive solution. Explain reasoning clearly so Learner can follow. Each round: correct Learner's mistakes and deepen explanation where it struggled.

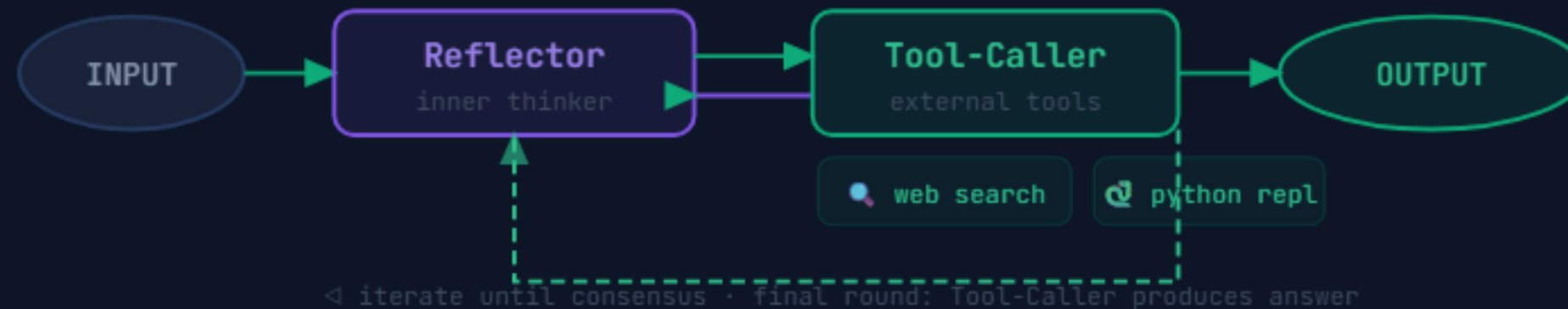
Learner Role

Attempt the problem using Expert's guidance. Identify uncertainty and flag it. Each round: internalise feedback and become more autonomous. Track improvement across rounds.

Deliberation Style tool-integrated

Reflector ↔ Tool-Caller

Inner thinker and external tool-user exchange, critique, and refine until reaching consensus. Only Tool-Caller has access to web search and Python execution.



Reflector

Reasons deeply without tools. Analyzes Tool-Caller's findings, forms hypotheses, identifies what's still uncertain, critiques candidate answers.

Tool-Caller

Retrieves external facts via DuckDuckGo search, runs Python code. Grounds final answer in evidence. Produces complete answer in last round.

OUR IMPLEMENTATION

Built with LangGraph + Claude

Each agent role is created with `create_react_agent`. The recursive loop is a `StateGraph` with conditional back-edges.

```
# rmas/agents/factory.py
from langgraph.prebuilt
import create_react_agent from langchain_anthropic
import ChatAnthropic
def create_agent(role,
description, tools=None): llm =
ChatAnthropic(model="claude-sonnet-4-6")
return
create_react_agent( model=llm, tools=tools or [],
prompt=build_prompt(role, description), )
```

```
# rmas/patterns/sequential.py
graph =
StateGraph(RMASState)
graph.add_node("planner", planner_node)
graph.add_node("critic", critic_node)
graph.add_node("solver", solver_node)
graph.add_edge("planner", "critic")
graph.add_edge("critic", "solver") # Conditional back-
```

SHARED STATE (RMASSTATE)

```
class RMASState(TypedDict): # Full conversation
history messages: Annotated[list, add_messages] #
Recursion tracking round: int max_rounds: int # Agent
"latent thoughts" context: dict[str, Any] # Final
decoded answer final_answer: str
```

`Context = RecursiveLink (text approximation)`

The **context** dict passes each agent's last output to all subsequent agents — simulating the outer `RecursiveLink` latent state transfer.

RUNNING THE APP

CLI Commands & Configuration

Set your `ANTHROPIC_API_KEY` in `.env`, then run any of the 4 patterns.

```
terminal · C:\Users\PLNAYAK\Documents\RMAS
```

```
# Sequential pattern (default, 2 rounds)
```

```
$ uv run main.py
```

```
# Mixture with 3 recursion rounds
```

```
$ uv run main.py --pattern mixture --rounds 3
```

```
# Distillation with custom question
```

```
$ uv run main.py --pattern distillation \
  --question "Explain attention"
```

```
# Deliberation (uses web search + Python)
```

```
$ uv run main.py --pattern deliberation
```

PYTHON API

```
from rmas import RecursiveMAS, RMASConfig from rmas
import CollaborationPattern config = RMASConfig(
pattern=CollaborationPattern.SEQUENTIAL,
recursion_rounds=3, model="claude-sonnet-4-6", ) rmas
= RecursiveMAS(config) result = rmas.run("Your
question here") print(result.final_answer)
```

[sequential](#)[mixture](#)[distillation](#)[deliberation](#)